

Module 10 Lab:

Bloodhound for AD Attack Path

JR Clark (002035589)

NEU DAMG7350 (Spring 2026)

Introduction

This lab explores how misconfigurations in Active Directory (AD) create privilege-escalation paths that attackers can exploit to gain unauthorized access to high-value roles such as Domain Admins. Using SharpHound for data collection and BloodHound CE for graph-based analysis, I built a multi-VM environment, captured AD relationships, and analyzed privilege paths for three accounts: admin, service-sql, and intern.

The final objective was to design and implement an alternative privilege escalation path, one not provided in the lab instructions, and demonstrate how BloodHound detects it. This report documents the environment, the escalation path I created, and the security implications of the misconfiguration.

Environment Setup Summary

The following components were configured:

- **Windows Server 2025** as the Domain Controller (LAN1-DC)
- **Active Directory Domain Services** and **Certificate Services**
- **Organizational Units:** *Admins, Service Accounts, Users*
- **Accounts:**
 - *admin* (Domain Admins)
 - *service-sql* (Domain Users)
 - *intern* (Domain Users → later modified)
- **Windows 11 VM** joined to the domain
- **Kali Linux VM** running BloodHound CE and Neo4j
- **SharpHound** executed multiple times to capture changes

A full step-by-step log with screenshots is included in the Appendix.

Baseline BloodHound Analysis

Before introducing any misconfigurations, I validated the expected privilege relationships:

1. **admin → Domain Admins** -- BloodHound correctly showed a direct membership path.

2. **intern** → **Domain Admins** -- BloodHound showed **no path**, confirming intern had no elevated privileges.

3. **intern** → **Account Operators** -- Also **no path**, establishing a clean baseline.

These baseline results were important for demonstrating how later misconfigurations changed the privilege graph.

Lab-Provided Escalation Example

Following the lab instructions, I implemented the instructor-guided misconfiguration:

- Added *service-sql* to **Domain Admins**
- Delegated **GenericAll** permissions on *service-sql* to *intern*

BloodHound correctly identified the escalation path: **INTERN** → **GenericAll** → **SERVICE-SQL** → **MemberOf** → **DOMAIN ADMINs**

This validated that SharpHound and BloodHound were functioning correctly.

Alternative Privilege Escalation Path (My Design)

To meet the final requirement—designing a *different* escalation path—I implemented a misconfiguration that is subtle, realistic, and commonly overlooked in enterprise AD environments:

Misconfiguration: Delegating Full Control of the Admins OU to intern

In ADUC:

1. Right-click **Admins OU**
2. Select **Delegate Control**
3. Add **intern**
4. Assign **Full Control** over the OU

This gives intern the ability to modify **any account inside the Admins OU**, including the *admin* account, which is a member of Domain Admins.

Why this is dangerous

Delegating control at the OU level is often overlooked because:

- It does **not** modify the Domain Admins group directly
- It does **not** appear when searching for “Domain Admins” in BloodHound
- It creates an **indirect** but fully exploitable escalation path

This is exactly the type of misconfiguration BloodHound is designed to reveal.

BloodHound Detection of the Alternative Path

After running SharpHound again and ingesting the new data, BloodHound CE revealed the escalation chain: **INTERN** → **GenericAll** → **Admins OU** → **Contains** → **ADMIN** → **MemberOf** → **DOMAIN ADMINs**

Interpretation

- **GenericAll on the OU** allows intern to reset the admin password or modify its group membership.
- Because *admin* is a Domain Admin, compromising it compromises the domain.
- This path is **indirect**, meaning it does not appear when searching for “Domain Admins” directly.
- Analysts must use **Outbound Object Control** or **Pathfinding** to discover it.

This demonstrates a realistic, high-impact privilege escalation vector.

Security Implications

This misconfiguration illustrates several key risks:

- **OU-level delegation is often misused** and can silently grant broad administrative power.
- **Indirect escalation paths are harder to detect** without graph-based tools.
- **Privilege boundaries in AD are porous** when ACLs are misconfigured.
- **Least privilege must apply to OUs**, not just groups and users.

In a real environment, this would be considered a **critical severity finding**.

Conclusion

This lab demonstrated how BloodHound CE can uncover both direct and indirect privilege-escalation paths in Active Directory. By designing and implementing an alternative misconfiguration—delegating full control of the Admins OU to the intern account—I showed how a seemingly small ACL change can create a complete escalation path to Domain Admins.

A key takeaway from this exercise is that **privilege escalation cannot be understood by looking only at a single user and a single endpoint**. AD permissions form a *graph*, not a hierarchy. A user may have no direct path to a high-value group, yet still be able to compromise it through:

- control of an OU that contains privileged accounts,
- ACL rights on intermediate objects,
- nested group memberships, or
- inherited permissions that are not obvious from the target object itself.

This means that **searching for “Domain Admins” alone will not reveal every risk**. Analysts must examine the *entire privilege surface* of a user — including outbound object control, delegated rights, and indirect relationships — to understand what that user can truly do.

BloodHound’s graph-based approach is essential for uncovering these hidden paths, and this lab illustrates why organizations must evaluate privilege holistically rather than relying on isolated checks.

This reinforces the importance of continuous privilege auditing, least-privilege design, and a deeper understanding of AD ACLs to prevent subtle misconfigurations from becoming domain-level compromises.

References

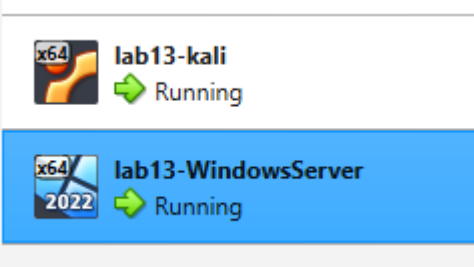
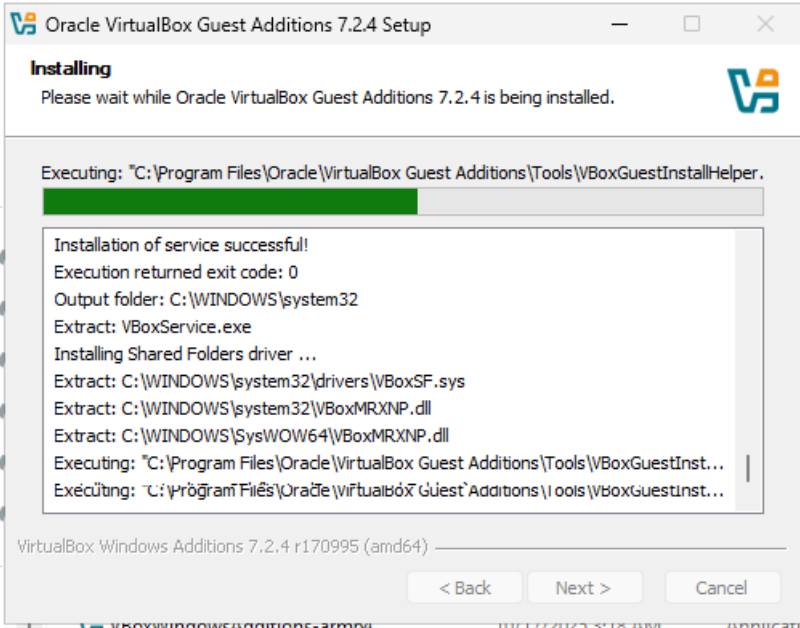
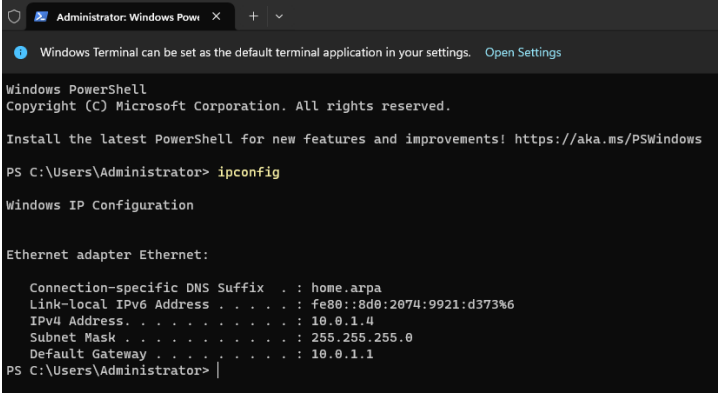
Sawant, A. (2026). Privilege Access Management [Online course]. Coursera & Northeastern University.

Methodology and Use of AI Tools

All technical steps were performed manually inside VirtualBox VMs (Kali Linux, Windows Server 2025, Windows 11). SharpHound collections were executed directly on the Windows 11 domain-joined workstation, and BloodHound CE was used on Kali for ingestion and analysis. Microsoft Copilot was used as a writing and organizational assistant to help structure the report, refine transitions, and ensure alignment with the assignment rubric. All technical actions, configurations, and interpretations reflect my own work and understanding.

APPENDIX A: Documented Process

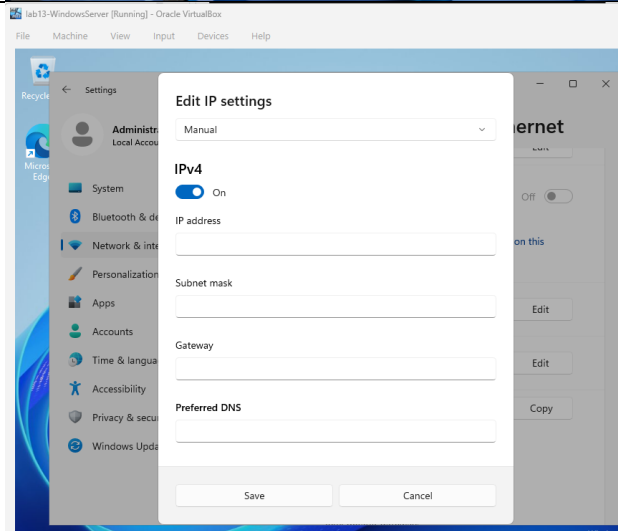
The following table documents the exact steps performed for this lab, with each left-column description corresponding to the screenshot or output shown in the right column.

Step 1 : Clone Kali Linux as lab13-Kali, download and create VM with Windows Server 2025	
Step 2: Get Windows Server setup and install VB Guest Additions	
Step 3: Turn on pfSense VM & check Windows Server IP address	Windows Server IP: 

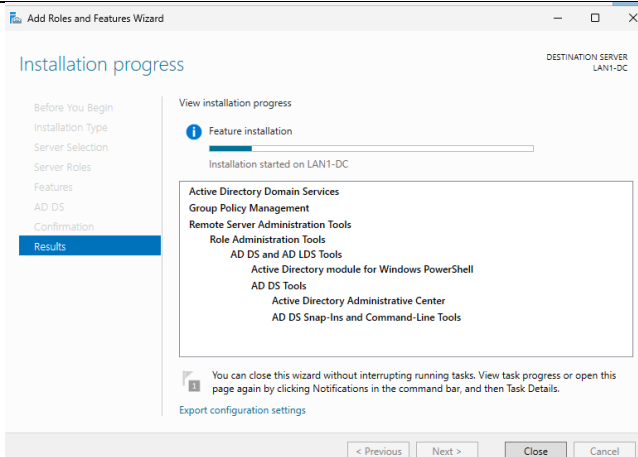
Step 4: acquire IP of eth1

```
jenn@kali-linux: ~  
Session Actions Edit View Help  
usage: sudo -l [-ABkNnS] [-g group] [-h host] [-p prompt] [-U user]  
[-u user] [command [arg ...]]  
usage: sudo [-ABbEHKnPS] [-r role] [-t type] [-C num] [-D directory]  
[-g group] [-h host] [-p prompt] [-R directory] [-T timeout]  
[-u user] [VAR=value] [-i l -s] [command [arg ...]]  
usage: sudo -e [-ABkNnS] [-r role] [-t type] [-C num] [-D directory]  
[-g group] [-h host] [-p prompt] [-R directory] [-T timeout]  
[-u user] file ...  
  
(jenn@kali-linux)-[~]  
$ sudo dhclient -v eth1  
[sudo] password for jenn:  
Internet Systems Consortium DHCP Client 4.4.3-P1  
Copyright 2004-2022 Internet Systems Consortium.  
All rights reserved.  
For info, please visit https://www.isc.org/software/dhcp/  
  
Listening on LPF/eth1/08:00:27:21:36:25  
Sending on LPF/eth1/08:00:27:21:36:25  
Sending on Socket/fallback  
DHCPOFFER on eth1 to 255.255.255.255 port 67 interval 5  
DHCPOFFER of 10.0.1.7 from 10.0.1.1  
DHCREQUEST for 10.0.1.7 on eth1 to 255.255.255.255 port 67  
DHCPCACK of 10.0.1.7 from 10.0.1.1  
bound to 10.0.1.7 -- renewal in 3046 seconds.  
  
(jenn@kali-linux)-[~]  
$
```

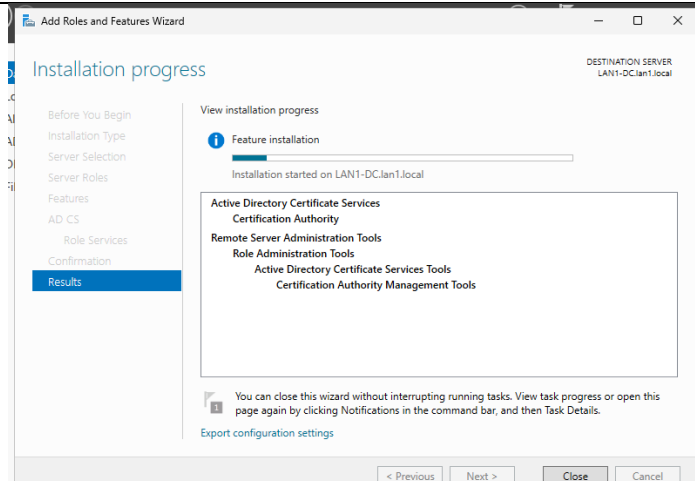
Step 5: set windows server IP to static, rename to LAN1-DC and reboot



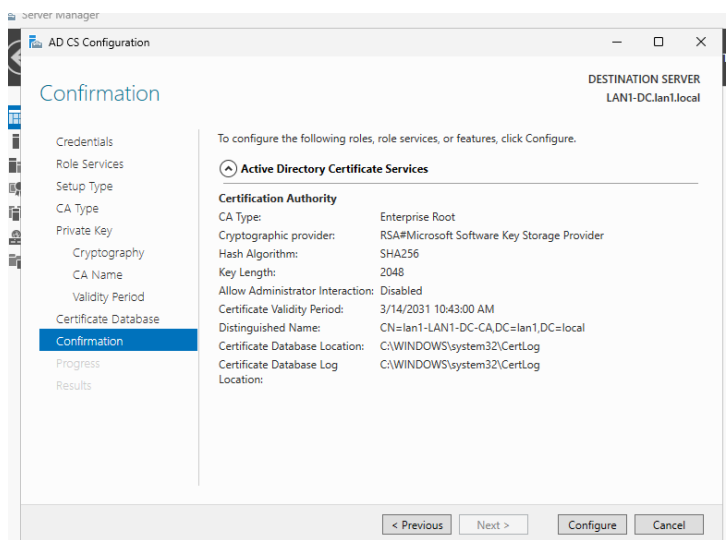
Step 6: Update Windows Server features



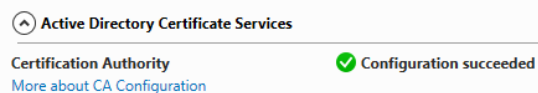
Step 7: Set up Active Directory – install and configure certificate services



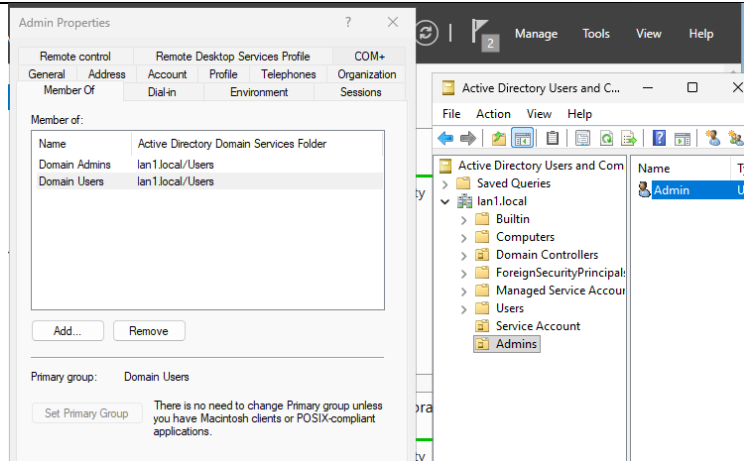
Step 8: Set up the certification authority

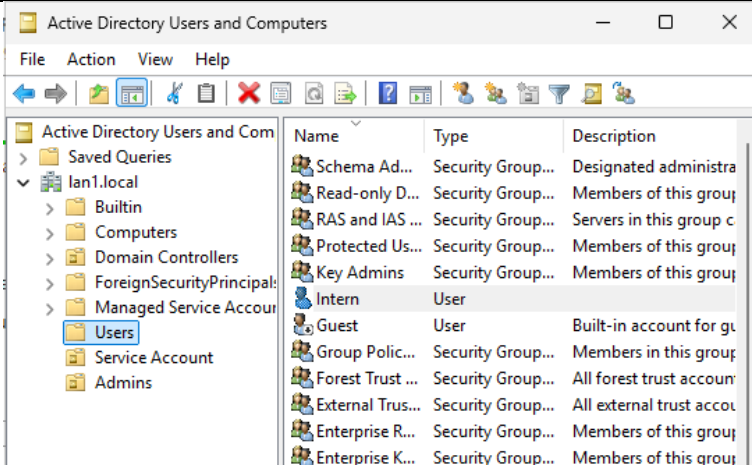
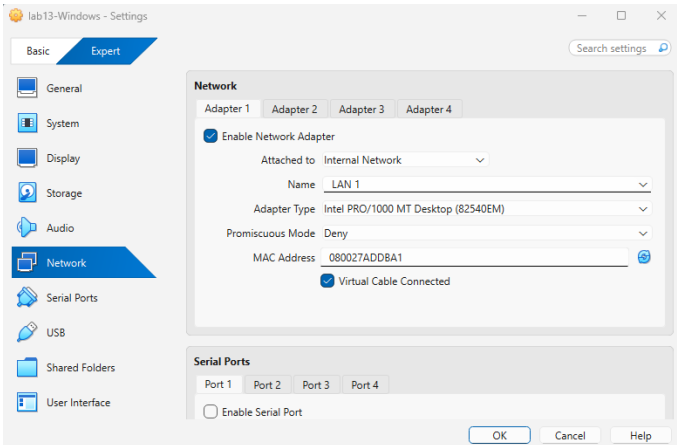
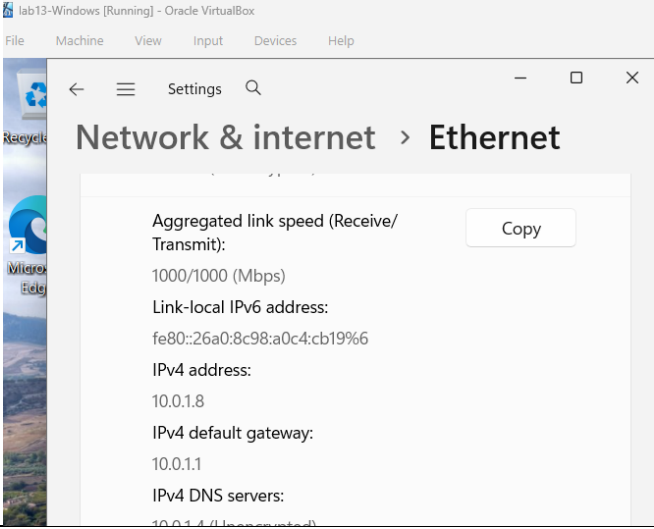


The following roles, role services, or features were configured:

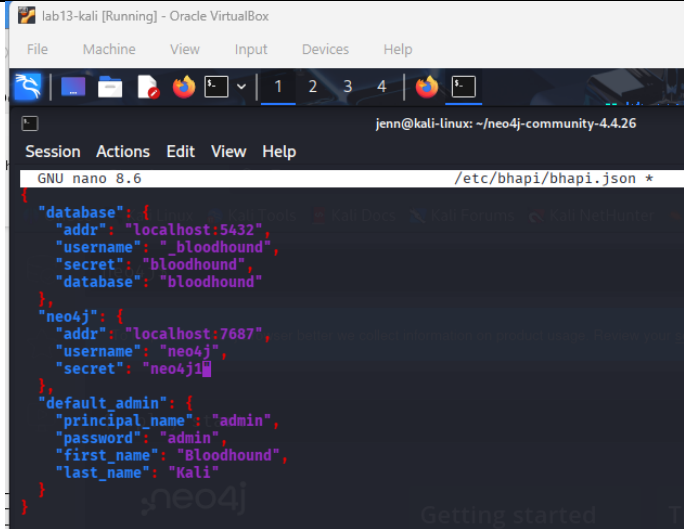


Step 9: configuring Admin user, adding service account and intern account



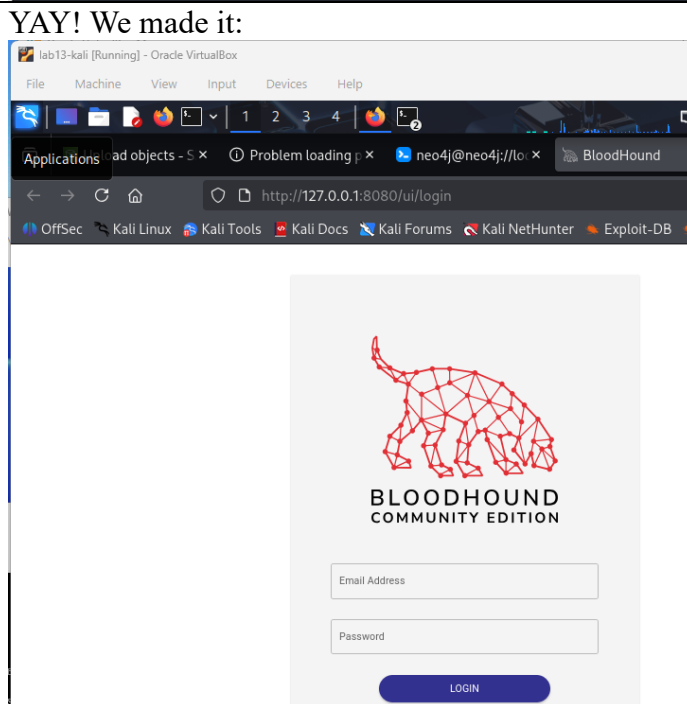
	
Step 10: Set up Windows11 VM and configure Network	
Step 11: Configure the Network settings for the Windows 11 VM so that it uses the IP of the Windows Server VM as the Preferred DNS	
Step 12: Install BloodHound on lab 13-kali	NOTE: This had multiple errors and after troubleshooting with CoPilot – I learned that bloodhound is part of the current install of Kali Linux:

Step 14: update the json file with the password

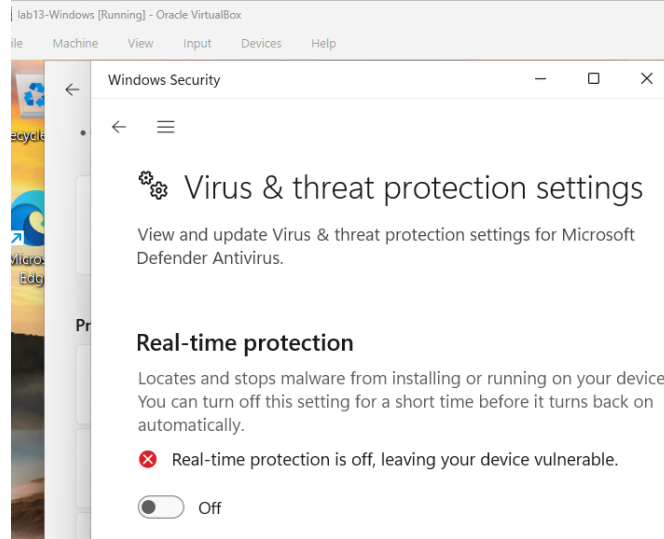


```
lab13-kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
jenn@kali-linux: ~/neo4j-community-4.4.26
Session Actions Edit View Help
GNU nano 8.6 /etc/bhapi/bhapi.json *
{
  "database": {
    "addr": "localhost:5432",
    "username": "bloodhound",
    "secret": "bloodhound",
    "database": "bloodhound"
  },
  "neo4j": {
    "addr": "localhost:7687",
    "username": "neo4j",
    "secret": "neo4j"
  },
  "default_admin": {
    "principal_name": "admin",
    "password": "admin",
    "first_name": "Bloodhound",
    "last_name": "Kali"
  }
}
```

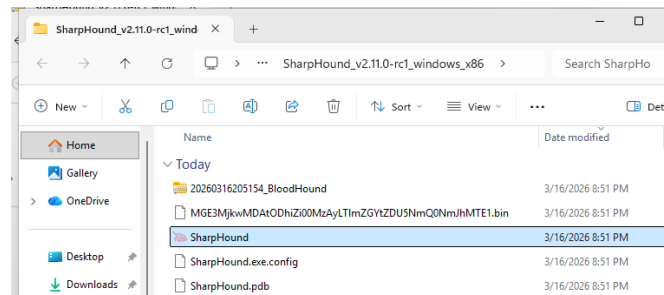
Step 15: start Bloodhound!



Step 16: Turn off virus & threat protection settings on the Windows 11 VM



Step 17: Install SharpHound and run – see resulting zipped file



Step 18: Transfer file to Kali VM

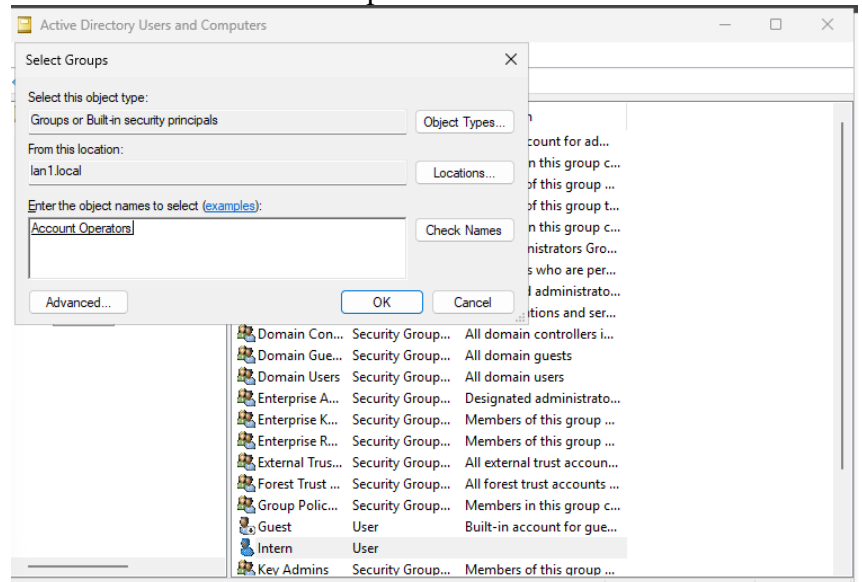
Note: dragging from the Windows 11 VM must be done to the Kali desktop not to the “file upload” dialog box on the console.

Step 19: Explore the data



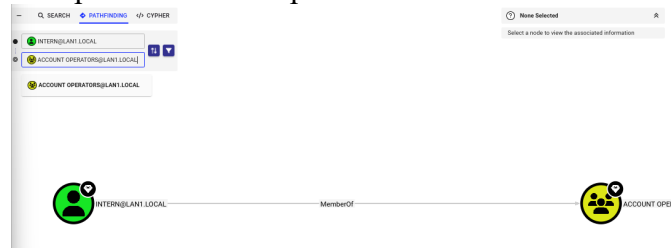
Step 20: Add more permissions to intern via the Windows Server VM

Intern added to “Account Operators”

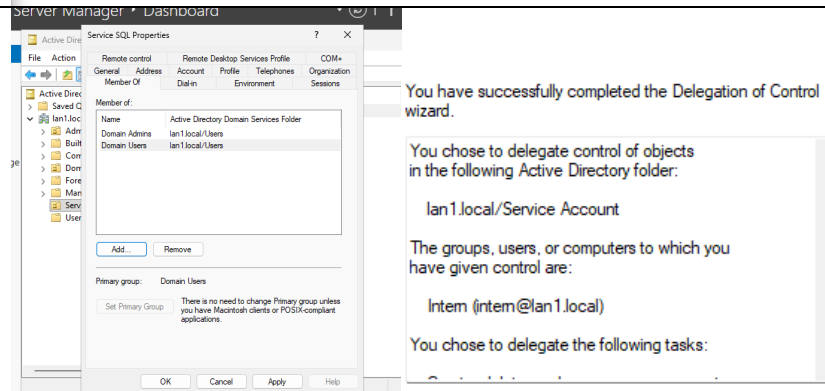


Step 21: Run Sharphound (Windows 11 VM) again and upload to Bloodhound (Kali VM)

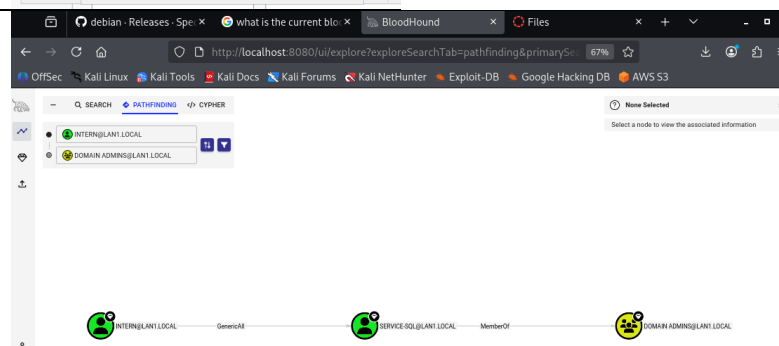
This shows the “explore” function demonstrating that the intern is now part of account operators:



Step 22: Add the Service SQL (on Windows Server VM) to “Domain Admins) & Delegate Control to intern



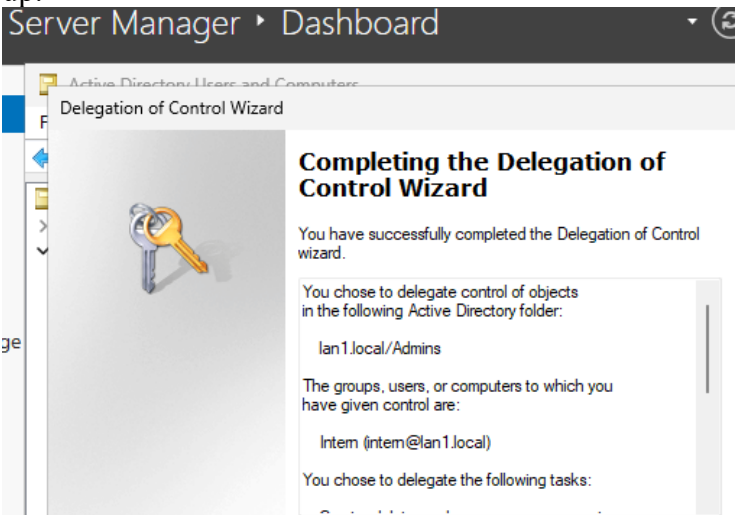
Step 23: Run Sharphound (Windows 11VM) and transfer that file to Kali VM for analysis in Bloodhound:



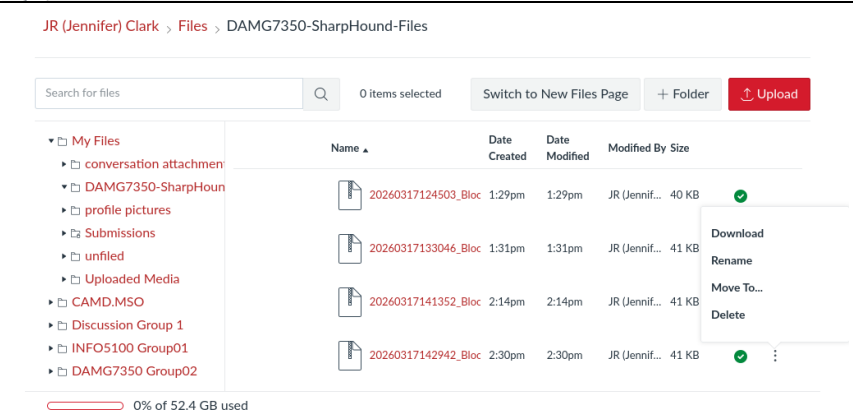
Step 24 (Step 76 in the lab document): create a misconfiguration that grants excessive permissions to intern.

Honestly, after spending nearly 20 hours dealing with repeated restarts, updates, crashes, rebuilding VMs and freezes – I want to remove all the WindowsDefender and password requirements from my VM.

If I was the intern in this situation – I would just go nuclear – give myself access as an Admin so I can fix things as they come up:



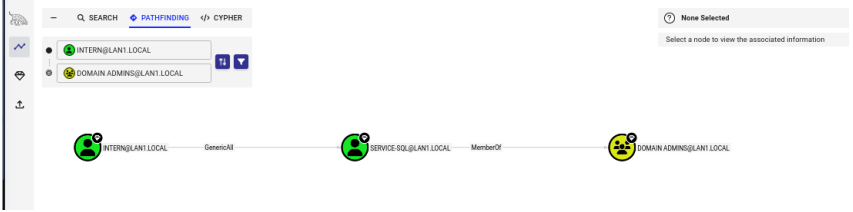
Step 25: Capture in SharpHound & analyze in BloodHound:



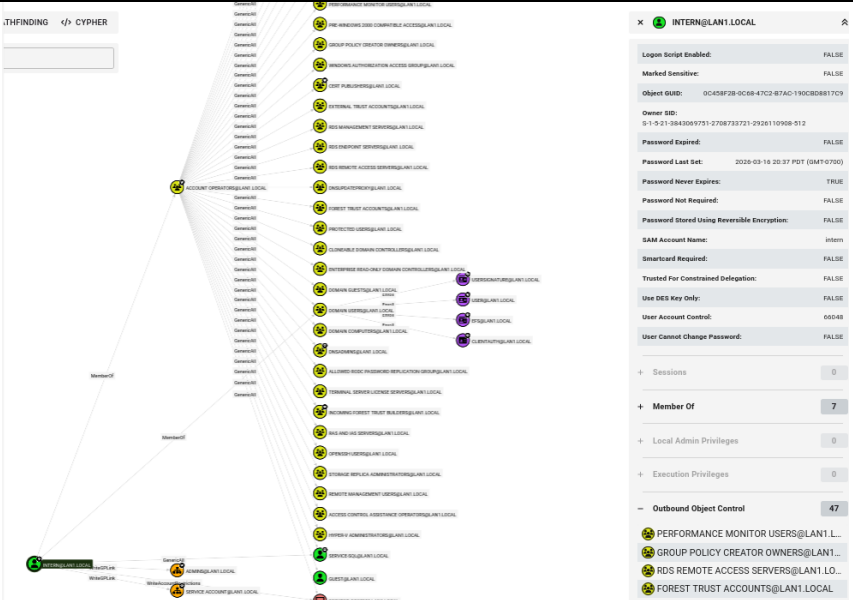
You can only see this escalation path by selecting “Admin@Lan1.local”



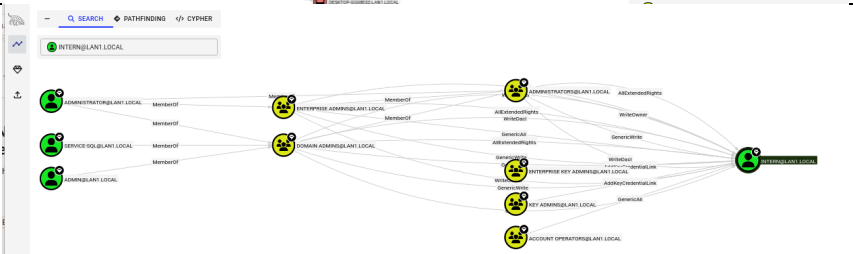
And you can see the existing path for Domain Admins here:



Step 26: Searching for known endpoints doesn't reveal all the risk for a particular user in privilege escalation. Using "Search" and checking "Outbound Object Control" will show all the items connected to that user:



Step 27: Inbound Object Control view:



Appendix B: Environment Failures, Rebuilds, and Rationale for Canvas-Hosted ISOs

This lab required the creation and configuration of multiple virtual machines (Windows Server 2025, Windows 11, Kali Linux, and pfSense), along with repeated installations of BloodHound, Neo4j, and SharpHound. While the technical concepts in the assignment were clear, the practical execution was significantly hindered by repeated failures, VM corruption, and environmental instability. These issues consumed more than four days of troubleshooting and rebuilding, and they highlight a structural problem with the current lab design.

Below is a consolidated record of the failures encountered and why they justify the need for **pre-built, instructor-provided VM images hosted on Canvas**.

Item	Impact
Repeated VM Rebuilds Due to Windows Server and Windows 11 Instability • Multiple forced restarts during installation caused partial or corrupted OS setups. • Windows Server frequently hung on “Applying Computer Settings,” requiring manual shutdowns. • Windows 11 updates triggered unexpected reboots that broke network configuration and domain membership. • Several VM states became unrecoverable, requiring full deletion and recreation.	Each rebuild required 1–2 hours of installation time, not including reconfiguration of AD, OUs, users, DNS, and domain joins.
Guest Additions Failures and Broken Integration Features • Guest Additions repeatedly failed to install or load correctly. • Clipboard sharing, drag-and-drop, and display resizing broke unpredictably. • Some failures were caused by kernel mismatches; others by Windows Defender blocking components.	These failures directly blocked file transfer workflows required by the lab (SharpHound → Kali), forcing workarounds and additional rebuilds.
Network Configuration Breakage Across Reboots • LAN1 internal network assignments were lost after VM restarts. • Windows Server occasionally reverted to DHCP despite manual static assignment.	BloodHound and SharpHound cannot function without a stable domain environment. Each network failure required re-diagnosis and reconfiguration.

Windows 11 sometimes failed to resolve the domain controller even with correct DNS settings.	
Neo4j and BloodHound Installation Failures - Neo4j service failed to start on first installation attempt (“service not found”). - BloodHound CE initially failed to launch due to missing dependencies. The lab instructions referenced older versions of BloodHound that no longer match the CE interface	Troubleshooting consumed hours and required multiple attempts before BloodHound CE successfully launched.
SharpHound Execution Blocked by Windows Defender - SharpHound binaries were repeatedly quarantined or blocked. - Even after disabling real-time protection, Windows Defender always re-enabled itself after reboots, and sometimes after screen-lock.	This forced repeated re-downloads, re-extractions, and re-runs of SharpHound.
File Transfer Failures Between VMs - Drag-and-drop only worked in one direction at a time and often failed silently. - Shared folders were unreliable due to Guest Additions instability. - Ultimately, the only reliable method was uploading files to Canvas from one VM and downloading them from another — a workaround discovered through trial and error.	This added unnecessary steps and highlighted the fragility of the environment. Ultimately, I ended up logging into Canvas and using the “Files” area to save these and transfer them between VMs. Instructions should tell students: if GuestAdditions doesn’t work after 1-2 attempts, just use Canvas or some other site (like Google drive) to move the files to reduce time wasted.

Why These Failures Justify Canvas-Hosted ISOs

The cumulative effect of these issues is that students spend the majority of their time:

- rebuilding VMs
- re-installing OSes
- re-configuring AD
- troubleshooting network failures
- fighting Windows Defender
- repairing broken integrations

Instead of learning **privilege access management**, **AD security**, and **BloodHound analysis**, which are the actual learning objectives.

Providing **pre-built, pre-tested VM images** on Canvas would:

- eliminate 80–90% of setup time
- ensure consistent environments across students
- prevent OS corruption and update-related failures
- remove dependency on unstable Guest Additions
- allow students to focus on AD security concepts rather than infrastructure repair
- reduce grading variability caused by environment issues rather than student understanding

This approach is already standard in cybersecurity education, where **reproducible environments are essential for learning**.

Recommendation

Northeastern University should provide **Canvas-hosted VM ISOs or OVA templates** for:

- Windows Server 2025 (pre-configured with AD DS and Certificate Services)
- Windows 11 (domain-join ready)
- Kali Linux (with BloodHound CE pre-installed)
- pfSense (pre-configured LAN1 network)

This would dramatically reduce student frustration, improve learning outcomes, and ensure that lab time is spent on **security analysis**, not **environment reconstruction**.